

Linear DS

[1, 2, 3, 4] -> [5, 1, 2, 3, 4]

Contiguous memory allocation -

1 -> x

2 -> x + 4

3 -> x + 8

4 -> x + 12

5 -> x

1 -> x + 4

2 -> x + 4 + 4

3 -> x + 8 + 4

4 -> x + 12 + 4

Insert at end: $O(1)$

Insert at the beginning: $O(n)$

Insert at somewhere in the middle: $O(n)$

Delete at end: $O(1)$

Delete at the beginning: $O(n)$

Delete at somewhere in the middle: $O(n)$

Arr[1000] -> 500th element access -> $O(1)$

0th element -> x

500th element -> $x + 499 \cdot 4$

Random access: $O(1)$

Another advantage -> Contiguous memory allocation makes the accessing elements in the array very cache-friendly.

Locality of references -> If you access some location in the memory, then it is very likely that you will access elements in the neighboring locations as well.

X -> also load some neighboring elements inside the cache.

1 -> x

2 -> x + 4

3 -> x + 8

4 -> x + 12

Want to insert at the end -> but x+16 is already occupied.

=> Array gets shifted to some other location `y` ->

1 -> y

2 -> y + 4

3 -> y + 8

4 -> y + 12

5 -> y + 16

2 -> 4 -> 8 -> 16 -> 32 -> 64 -> 128 -> 256 -> 512 -> 1024

Smallest Positive Number Missing from the Array

[3, 4, 5] -> 1

[1, 2, 3, 4, 5] -> 6

[0, -10, 1, 3, -20] -> 2

Brute Force:

[5, 2, 1, 3, 4] -> 6

result = 1 -> 2 -> 3 -> 4 -> 5 -> 6

TC: $O(n^2)$

AS: $O(1)$

Binary Search Approach:

[0, -10, 1, 3, -20] -> [-20, -10, 0, 1, 3]

Same as BF but instead of linear search, do binary search

TC: $O(n * \log(n))$

AS: $O(1)$

Another Sorting Approach:

[0, -10, 1, 3, -20] -> [-20, -10, 0, 1, 3]

Iterate to the first positive number and then contiguously search for other positive numbers.

TC: $O(n * \log(n) + n) = O(n * \log(n))$

AS: $O(1)$

[-20, -10, 0, **1**, 3], result = 1

[-20, -10, 0, 1, **3**], result = 2

Hashmap-Based Approach:

[0, -10, 1, 3, -20]

{Positive Numbers -> true/false}

```
{  
    1->true,  
    3->true,  
}
```

Result = 1 -> 2

TC: $O(n)$

AS: $O(n)$

Bucketing Technique Based Approach:

Search space for result -> [1, n + 1] for an array of size n

Size of the array -> n

If we know what positive numbers exist in [1, n], we will be able to tell the answer.

(0, n-1)

We will be using the array indices as **buckets** to store the information regarding what all positive numbers exist in the array.

```
0 1 2 3 4 5  
[ 4, 2, 1, 2, 4, 3 ]
```

For a positive number x -> we can simply use the index (x-1) store the information whether x is present in the array or not.

Index 0 will tell us whether 1 is present or not.

Index 1 will tell us whether 2 is present or not

Index 2 will tell us whether 3 is present or not

...

Index n-1 will tell us whether n is present or not

(Assume that the array contains only positive numbers).

0 1 2 3 4 5
[-4, -2, -1, -2, 4, 3]

0 1 2 3 4 5
[-4, -2, -1, -2, 4, 3] -> 5 is not present in the array.

TC: $O(n)$

AS: $O(1)$

Step-0: Convert all the non-positive numbers to n+1.

0 1 2 3 4
[-6, 6, -1, 3, 6], n = 5

Boolean Matrix Question

$n \times n$ matrix for simplicity

Input:

```
1 0 0 1
1 1 1 1
0 1 1 1
1 1 1 1
```

Output:

```
0 0 0 0
0 0 0 1
0 0 0 0
0 0 0 1
```

Brute Force:

In-place updates -> but instead of updating with 0s, update with 2s.

```
2 0 2
1 2 1 ->
1 2 1
```

```
0 0 0
1 0 1
1 0 1
```

```
2 0 2
1 2 1 ->
2 0 2
```

```
0 0 0
1 0 1
0 0 0
```

TC:

A single in-place update takes $O(n)$ time

Number of in-place updates in the worst case: $O(n^2)$

Overall TC: $O(n^3)$

AS: $O(1)$

Extra Space Approach:

1 0 1

1 1 1

1 0 1

rows[] = {true, false, true}

cols[] = {false, true, false}

0 0 0

1 0 1

0 0 0

TC: $O(n^2)$

AS: $O(n)$

Bucketing Technique:

(Assuming that the 0th row and the 0th col originally contain only 1s).

1 1 1 1 1

1 0 1 1 1

1 1 1 1 1

1 1 1 0 0

1 1 1 1 1

```
1 0 1 0 0
0 0 0 0 0
1 0 1 0 0
0 0 0 0 0
1 0 1 0 0
```

TC: $O(n^2)$

AS: $O(1)$

Input:

```
1 0 1
1 1 1
1 1 1
```

zerothRowContainsZero = true

zerothColContainsZero = false

Output:

```
0 0 0
1 0 1
1 0 1
```

Two booleans telling us initially whether the 0th row contains a zero
Or the 0th col contains a zero